C++ Fundamentals: The Core Architecture

A visual guide to fast, procedural programming.

[OK] Core Components Loaded

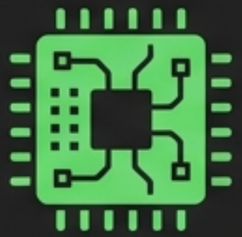
[OK] Core Components Loaded

Course Architecture: The 4 Modules

Module 1
Core Foundations
Syntax, Variables, & I/O.

Module 2
Control Flow
Logic, Decisions, & Loops.

Module 3
Functions
Reusable Code & Operations.



Module 4
Memory & Structures
Pointers, Arrays, & Custom Types.

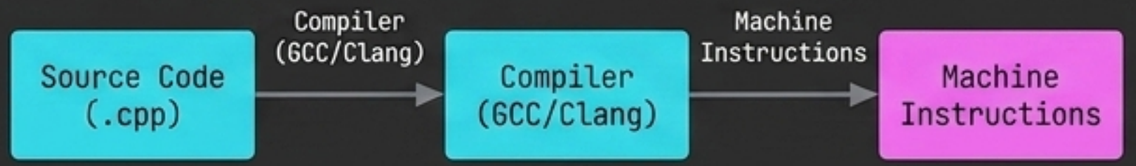
Module 1 | Anatomy of a C++ Program

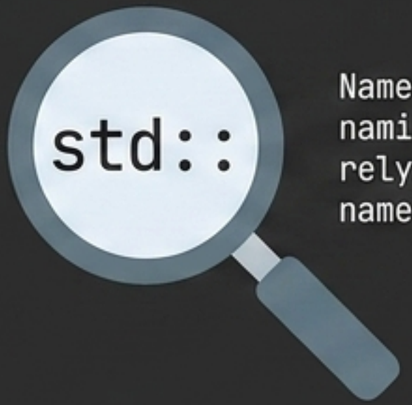
```cpp
#include <iostream>

int main() {
    std::cout << "Hello World";

    return 0;
}
```

- `#include <iostream>` — Imports basic input/output operations.
- `int main() {` — The program's starting point.
- `std::cout << "Hello World";` — Standard Character Output (with << insertion operator).
- `return 0;` — Exits without errors (returning 1 indicates an issue).





Namespaces: std:: prevents naming conflicts without relying on the risky 'using namespace std;'.

Data Types & Sizing Constraints

Integer Truncation

Type	Size (sizeof())	Example	Best Use Case
bool	1 byte	true	Two-state logic (Power on/off)
char	1 byte	'A'	Single characters/symbols
int	4 bytes	21	Whole numbers (Ages, years)
double	8 bytes	10.99	Decimals (Prices, GPA)
std::string	32 bytes	"Pizza"	Sequences of text

Prefix with 'const' to make a variable read-only (e.g., const double PI = 3.14159;).

Operations & Type Conversion

Modulus Trick

% returns the division remainder.

number % 2 == 0 means Even.
number % 2 == 1 means Odd.

Explicit Casting

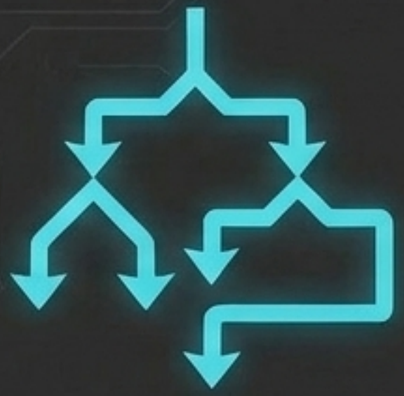
~~8 / 10 = 0~~

```
double score = (double)correct /
                    questions * 100;
```

Integer division truncates decimals. To retain percentages, explicitly cast the data type by placing it in parentheses.

Module 2 | Decision Making Matrix

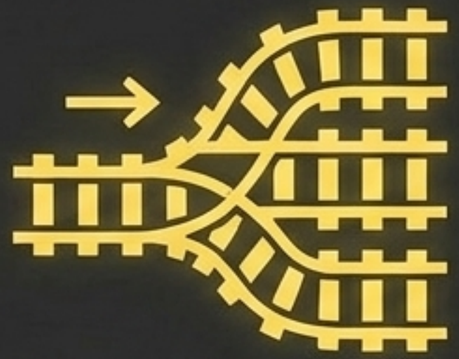
If/Else



Best for: Complex, range-based logic.

```
if (score >= 60) {
  cout << "Pass";
} else {
  cout << "Fail";
}
```

Switch

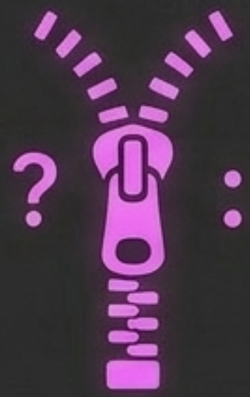


Best for: 1-to-1 matching against specific cases.

Rule: Must include 'break;' to prevent fall-through, and 'default:' for invalid inputs.

```
switch(grade) {
  case 'A': cout << "Pass"; break;
}
```

Ternary Operator (?:)

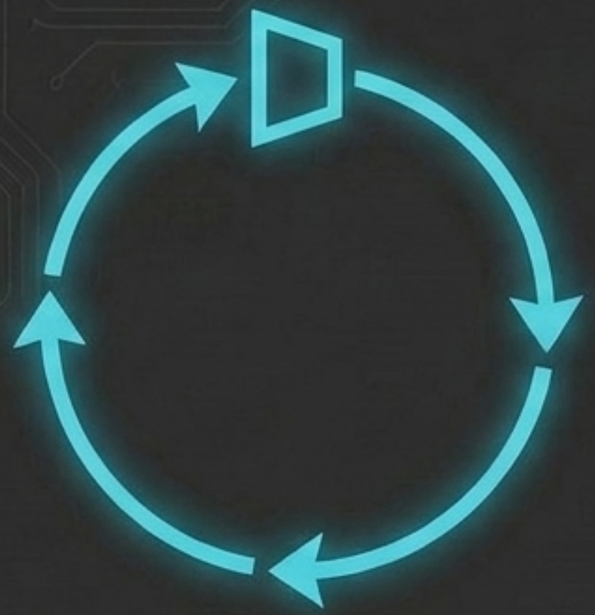


Best for: Inline, binary conditions.

```
score >= 60 ? cout << "Pass"
    : cout << "Fail";
```

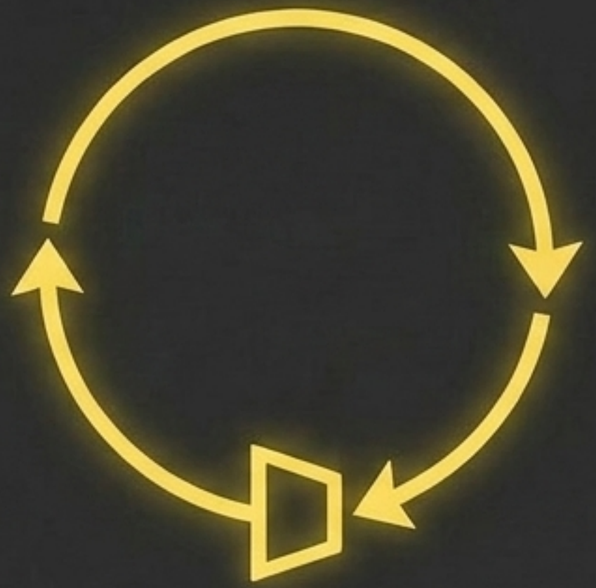
The Loop Architecture

While



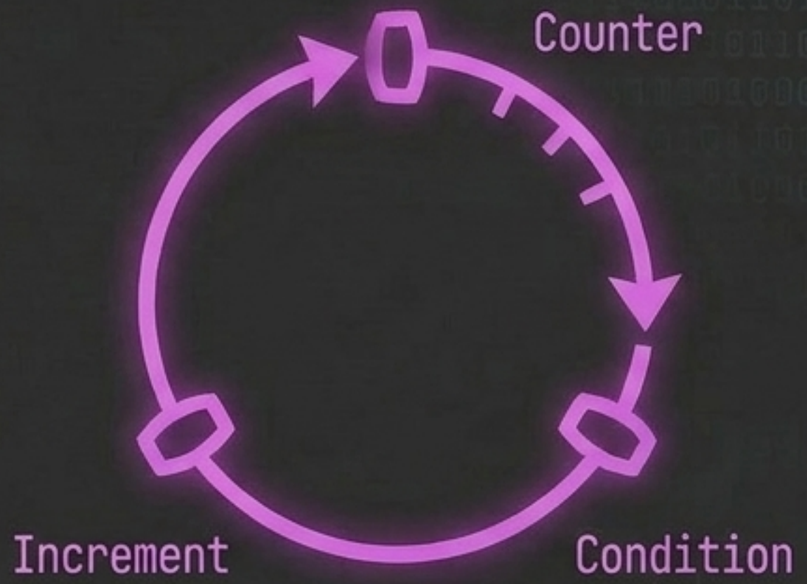
Condition evaluated first. Good for forcing valid user input.

Do-While



Code executes at least once before evaluating. Good for 'Play Again?' game prompts.

For

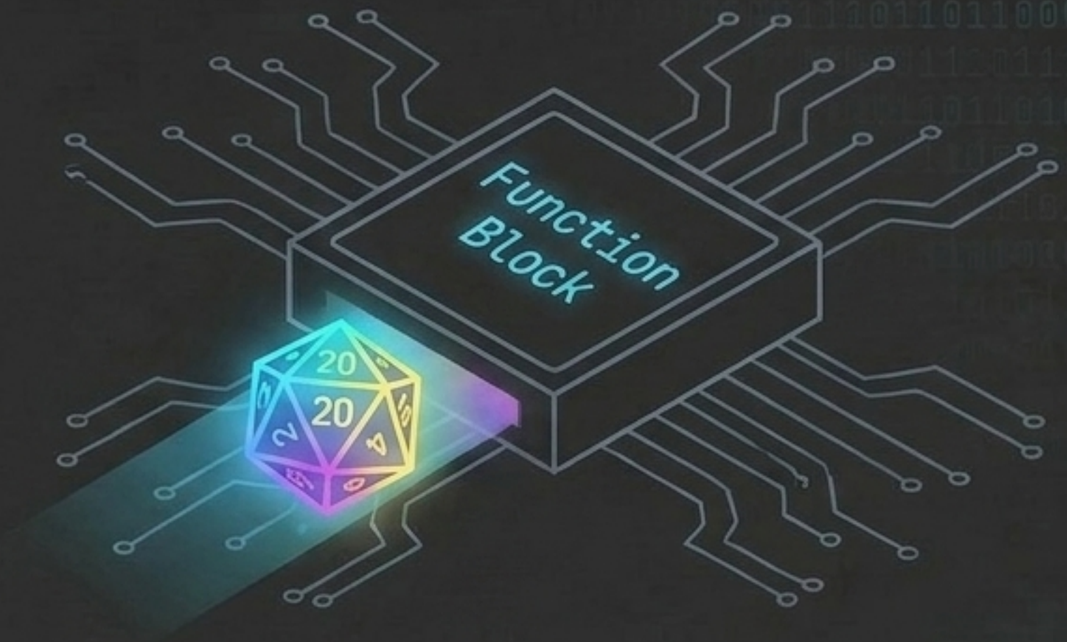


Best when the exact number of iterations is known (for(int i = 0; i <= 10; i++).

Loop Control: 'break' completely exits the loop. 'continue' skips the current iteration.

Logic Gates & Randomization

&& (AND)	: Both conditions must be true.
|| (OR)	: At least one condition must be true.
! (NOT)	: Reverses the logical state.

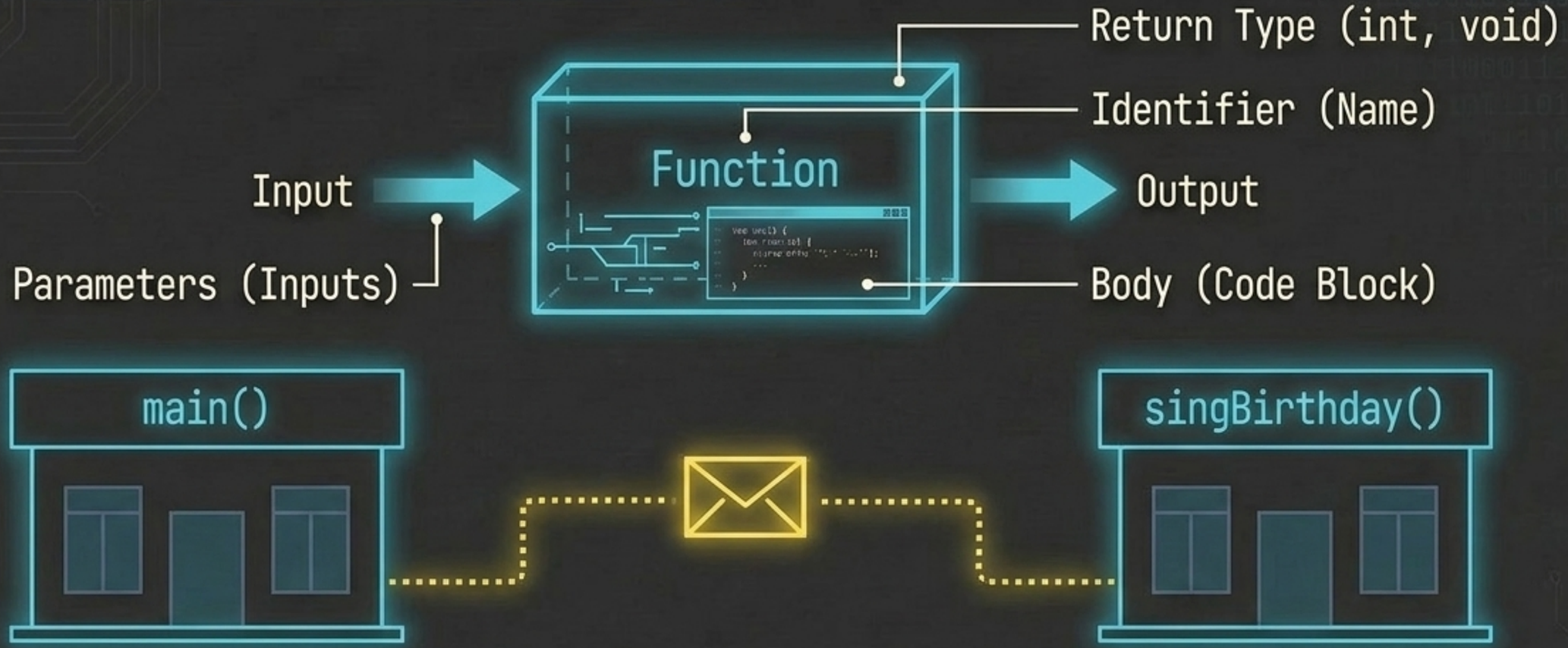


Pseudo-Random Generation

The Seed: 'srand(time(0));' initializes the generator using the current calendar time.

The Roll: 'int num = rand() % 20 + 1;' generates a random event between 1 and 20.

Module 3 | Function Anatomy & Scope

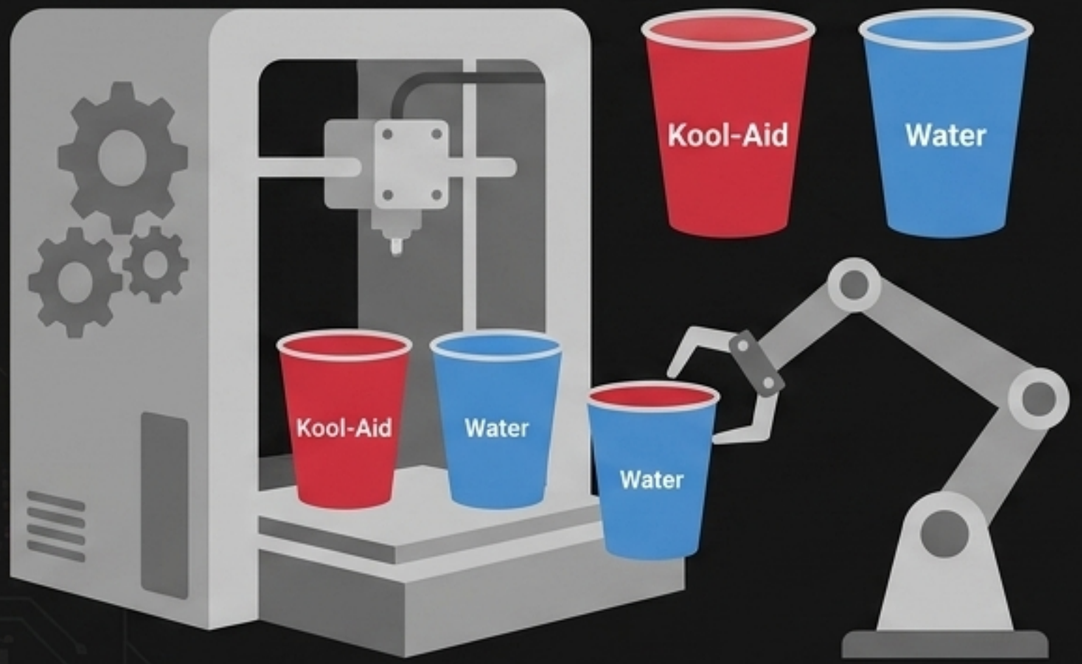


Variable Scope: Functions cannot see inside other functions. A local variable declared in `main()` is invisible to adjacent functions.

The Bridge: To share data, you must pass the variable as an **argument**, which the receiving function catches in a matching parameter.

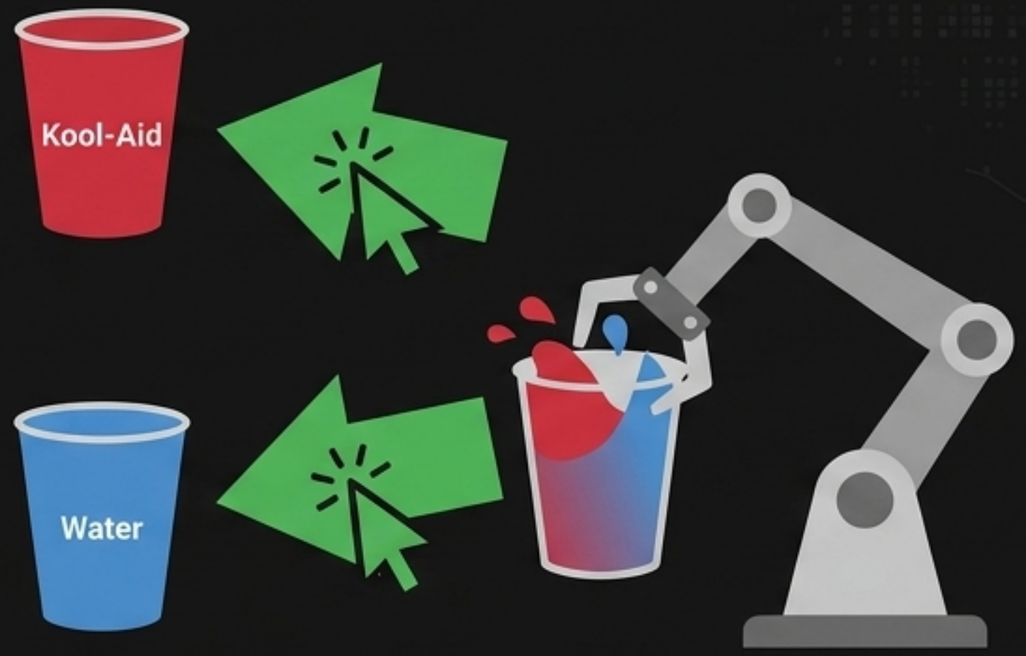
Memory Mechanics: Value vs. Reference

Pass by Value



Passes a copy of the variable. Swapping or altering the arguments inside the function does not affect the original data. Safe, but uses more memory.

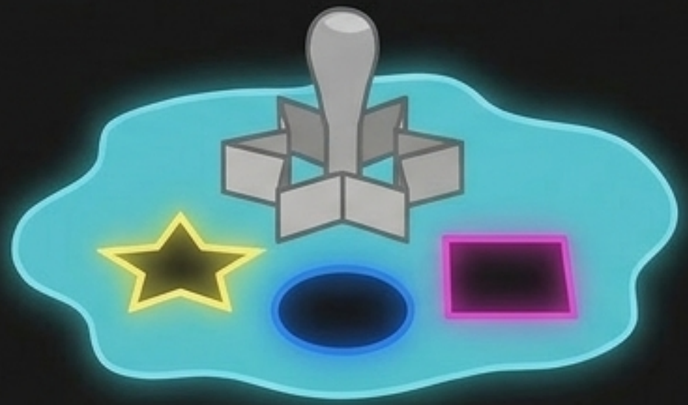
Pass by Reference (&)



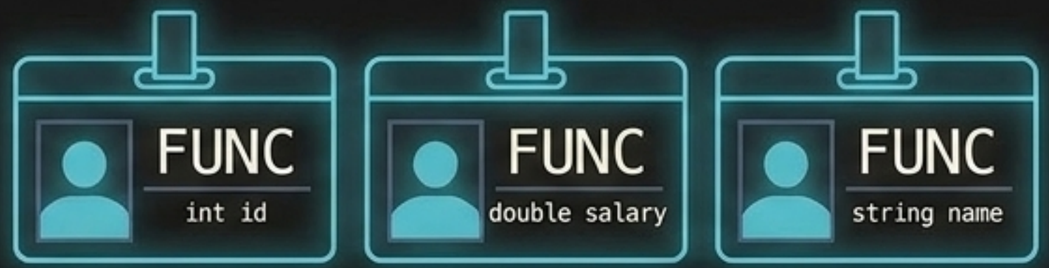
Passes the memory address of the original variable. Alters the actual data. Highly memory-efficient.

Advanced Function Architecture

Recursion: A function calling itself.
Warning: Without a proper base-case, it will cause an infinite loop and a Stack Overflow.



Function Templates: A 'cookie cutter' for data types. Using 'template <typename T>', one function can dynamically accept int, double, or char without writing redundant code.



Overloaded Functions: Functions sharing the exact same name, but utilizing different parameter signatures (e.g., bakePizza() vs bakePizza(string topping)).

Built-in Methods Dashboard

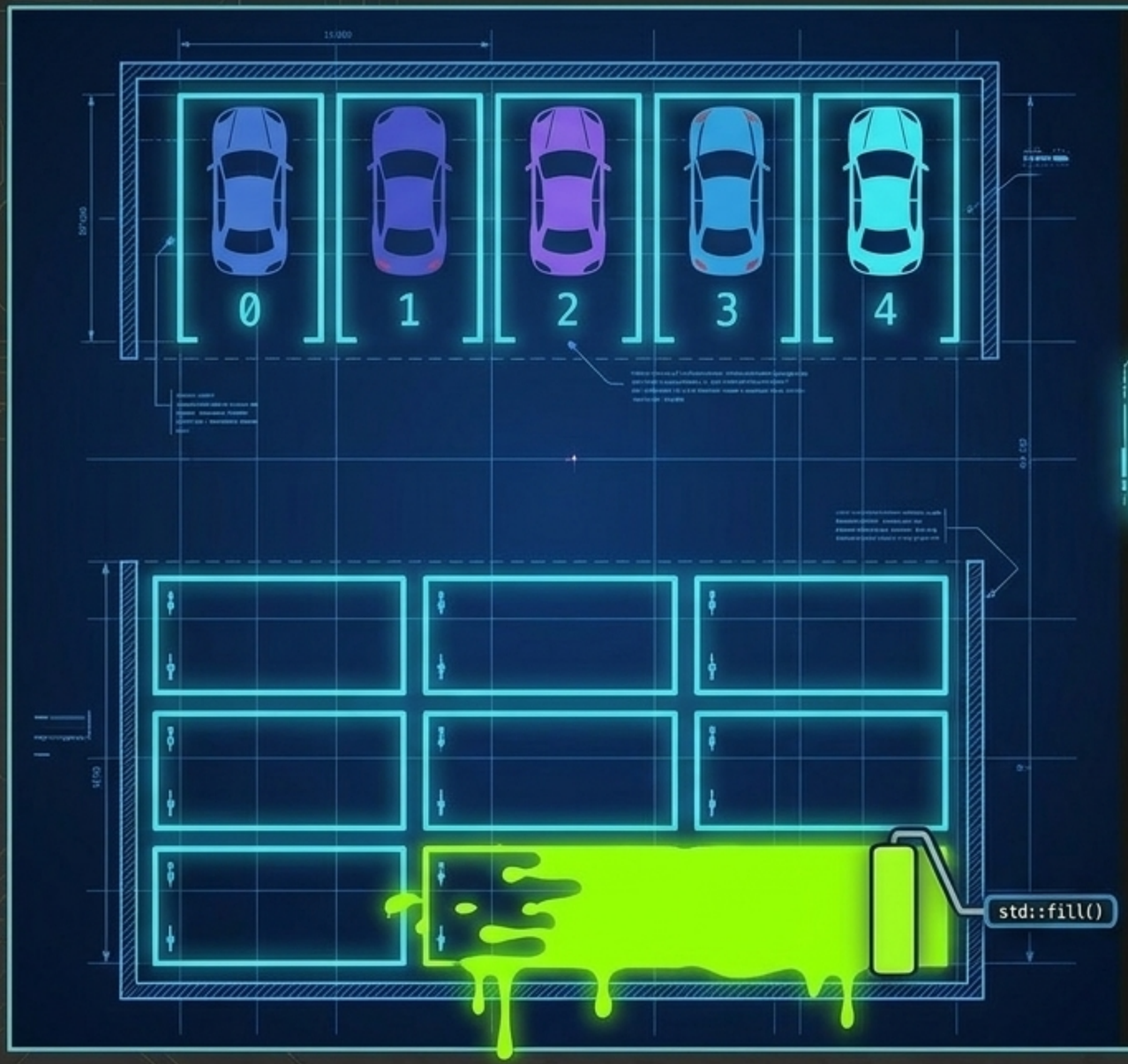
<cmath>

`max(x, y) / min(x, y)`	-> Returns greatest/least value.
`pow(2, 3)`	-> Returns 8 (2^3).
`sqrt(9)`	-> Returns 3.
`ceil(3.14)`	-> Returns 4 (rounds up).
`floor(3.99)`	-> Returns 3 (rounds down).

<string>

`name.length()`	-> Returns character count.
`name.empty()`	-> Returns boolean true/false.
`name.insert(0, "@")`	-> Inserts char at index position.
`name.find(" ")`	-> Returns index of first space.
`name.erase(0, 3)`	-> Deletes subset of characters.

Module 4 | Arrays & Contiguous Memory



1D Arrays: Data structures storing multiple values of the same type in adjacent memory blocks. Always indexed starting at 0.

Dynamic Iteration Formula:
`sizeof(array) / sizeof(array[0])`

Arrays are static in size. Use this formula to safely iterate without hardcoding lengths.

The Fill Function: 'std::fill(begin, end, value)' rapidly populates a range of elements without requiring a manual 'for' loop.

The Memory Map: Addresses & Pointers

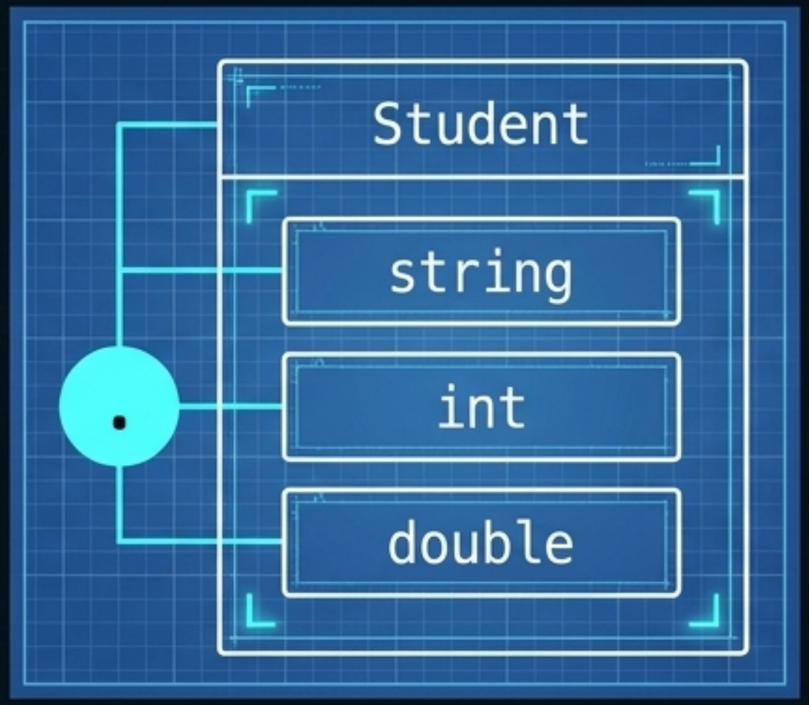
Address-of (&): Grabs the exact hexadecimal coordinate of where a variable 'lives' in RAM.

Pointers (*): Variables that store memory addresses rather than data.

Dereference (*): Traveling to the pointer's coordinate to access or alter the data inside the building.

Dynamic Memory: Use 'new' to allocate memory on the Heap during runtime, and 'delete' to prevent memory leaks. Unassigned pointers should default to 'nullptr'.

Custom Groupings: Structs & Enums



Structs: A data structure grouping related variables of different data types under a single name (e.g.,a Student containing a string name and double GPA). Members are accessed via the class member access operator (dot .).

Enums (Enumerations): A user-defined type consisting of paired named integer constants. Essential for making code highly readable, especially when used as 'switch' statement cases instead of arbitrary numbers.

Conclusion: With memory control, logic gates, and custom structs, you hold the complete procedural architecture of C++.